```
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.datasets import load_breast_cancer
dataset = load_breast_cancer()
X = dataset.data
y = dataset.target
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=3)
```

```
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn.score(X_test, y_test)
```

```
0.916083916083916
```

```
bag_knn=BaggingClassifier(KNeighborsClassifier(n_neighbors=5),
                          n_estimators=10,
                          max_samples=0.5,
                          bootstrap=True,
                          oob_score=True)
```

```
bag_knn.oob_score_
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_4738/3440012275.py in <cell line: 0>()
----> 1 bag_knn.oob_score_

AttributeError: 'BaggingClassifier' object has no attribute 'oob_score_'
```

```
bag_knn.fit(X_train, y_train)
bag_knn.score(X_test, y_test)
```

```
0.9300699300699301
```

```
pasting_knn=BaggingClassifier(KNeighborsClassifier(n_neighbors=5),
                          n_estimators=10,
                          max_samples=0.5,
                          bootstrap=False,
                          random_state=3)
```

```
pasting_knn.fit(X_train, y_train)
pasting_knn.score(X_test, y_test)
```

```
0.9300699300699301
```

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.ensemble import RandomForestClassifier
from sklearn import tree

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    roc_curve,
    roc_auc_score
)

from io import StringIO   # Correct replacement

from IPython.display import Image

import pydotplus
```

```
data = pd.read_csv("/content/sample_data/winequality-red.csv")
data
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68	9.8	5
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65	9.8	5
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58	9.8	6
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5
...	...	...	...	...	...	...	...	...	...	...	...	...
1591	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5
1592	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6
1593	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6
1594	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5
1595	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0	6

```
data.describe()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	su
count	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596
mean	8.314160	0.527954	0.270276	2.535558	0.087120	15.858396	46.382206	0.996744	3.311917	0
std	1.732203	0.179176	0.193894	1.405515	0.045251	10.460554	32.839138	0.001888	0.153346	0
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.860000	0
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000000	0.995600	3.210000	0
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000	0.996745	3.310000	0
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000000	0.997833	3.400000	0
max	15.600000	1.580000	0.790000	15.500000	0.611000	72.000000	289.000000	1.003690	4.010000	1

```
X=data.drop(columns='quality')
y=data['quality']
```

```
x_train , x_test , y_train , y_test = train_test_split(X,y,test_size=0.30,random_state=355)
```

```
clf=DecisionTreeClassifier(min_samples_split=2)
clf.fit(x_train,y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

```
clf.score(x_test,y_test)
```

```
0.5991649269311065
```

```
clf2=DecisionTreeClassifier(criterion='entropy',max_depth=24,min_samples_leaf=1)
clf2.fit(x_train,y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=24)
```

```
clf2.score(x_test,y_test)
```

```
0.5949895615866388
```

```
rand_clf=RandomForestClassifier(random_state=6)
```

```
rand_clf.fit(x_train,y_train)
rand_clf.score(x_test,y_test)
```

```
0.6805845511482255
```

```
grid_param = {
    "n_estimators" : [90,100,115,130],
    'criterion': ['gini', 'entropy'],
    'max_depth' : range(2,20,1),
    'min_samples_leaf' : range(1,10,1),
    'min_samples_split': range(2,10,1),
    'max_features' : ['auto','log2']
}
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
```

```
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
rand_clf = RandomForestClassifier(random_state=42)
```

```
grid_param = {
    'n_estimators': [50, 100],
    'max_depth': [5, 10],
    'min_samples_split': [2, 5]
}
```

```
random_search = RandomizedSearchCV(
    estimator=rand_clf,
    param_distributions=grid_param,
    n_iter=20,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)
```

```
random_search.fit(x_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:317: UserWarning: The total space of parameters 8
  warnings.warn(
Fitting 3 folds for each of 8 candidates, totalling 24 fits
```

▸ **RandomizedSearchCV**

▸ **best_estimator_: RandomForestClassifier**

▸ RandomForestClassifier

```
print("Best Parameters:", random_search.best_params_)
```

```
Best Parameters: {'n_estimators': 50, 'min_samples_split': 2, 'max_depth': 5}
```

```
grid_search = GridSearchCV(
    estimator=rand_clf,
    param_grid=grid_param,
    cv=3,
    verbose=2,
    n_jobs=-1
)
```

```
grid_search.fit(x_train, y_train)
```

```
Fitting 3 folds for each of 8 candidates, totalling 24 fits
```

GridSearchCV

best_estimator_: RandomForestClassifier

RandomForestClassifier

```
grid_search.best_params_
```

```
{'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 50}
```

```
rand_clf = RandomForestClassifier(criterion='entropy',
      max_depth=12,
      max_features='sqrt',
      min_samples_leaf=1,
      min_samples_split=2,
      n_estimators=90,
      random_state=42)
```

```
rand_clf.fit(x_train,y_train)
```

RandomForestClassifier

```
RandomForestClassifier(criterion='entropy', max_depth=12, n_estimators=90,
                       random_state=42)
```

```
rand_clf.score(x_test,y_test)
```

```
0.5666666666666667
```

```
grid_param={
    'n_estimators':[90,100,115],
    'criterion':['gini','entropy'],
    'min_samples_leaf':[1,2,3,4,5],
    'min_samples_split':[4,5,6,7,8],
    'max_features':['sqrt','log2']
}
```

```
grid_search = GridSearchCV(estimator=rand_clf, param_grid=grid_param, cv=3, verbose=2, n_jobs=-1)
grid_search.fit(x_train,y_train)
```

Show hidden output

```
grid_search.best_params_
```

```
{'criterion': 'gini',
 'max_features': 'sqrt',
 'min_samples_leaf': 3,
 'min_samples_split': 4,
 'n_estimators': 90}
```

```
rand_clf = RandomForestClassifier(criterion='entropy',
        max_features='sqrt',
        min_samples_leaf=1,
        min_samples_split=4,
        n_estimators=115,
        random_state=6)
```

```
rand_clf.fit(x_train,y_train)
```

RandomForestClassifier

```
RandomForestClassifier(criterion='entropy', min_samples_split=4,
                       n_estimators=115, random_state=6)
```

```
rand_clf.score(x_test,y_test)
```

```
0.5333333333333333
```

```
import pickle

with open('modelForPrediction.sav', 'wb') as f:
```

```
    pickle.dump(rand_clf, f)
```

```
import pandas as pd
data = pd.read_csv("/content/sample_data/california_housing_train.csv")
data.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income
count	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000
mean	-119.562108	35.625225	28.589353	2643.664412	539.410824	1429.573941	501.221941	3.883578
std	2.005166	2.137340	12.586937	2179.947071	421.499452	1147.852959	384.520841	1.908157
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900
25%	-121.790000	33.930000	18.000000	1462.000000	297.000000	790.000000	282.000000	2.566375
50%	-118.490000	34.250000	29.000000	2127.000000	434.000000	1167.000000	409.000000	3.544600
75%	-118.000000	37.720000	37.000000	3151.250000	648.250000	1721.000000	605.250000	4.767000
max	-114.310000	41.950000	52.000000	37937.000000	6445.000000	35682.000000	6082.000000	15.000100

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn import tree
from sklearn.model_selection import train_test_split
import numpy as np
```

```
data = pd.read_csv("/content/diabetes (1).csv")
data
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

Next steps: Generate code with data | New interactive sheet

```
x= data.drop(columns='Outcome')
y= data['Outcome']
```

```
train,val_train,test,val_test = train_test_split(x,y,test_size=0.5,random_state=355)
```

```
x_train,x_test,y_train,y_test = train_test_split(train,test,test_size=0.2,random_state=355)
```

```
knn= KNeighborsClassifier()
knn.fit(x_train,y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
knn.score(x_test,y_test)
```

```
0.7402597402597403
```

```
svm = SVC()
svm.fit(x_train,y_train)
```

SVC

SVC()

```
svm.score(x_test,y_test)
```

0.7402597402597403

```
predict_val1 = knn.predict(val_train)
predict_val2 = svm.predict(val_train)
```

```
predict_val1=np.column_stack((predict_val1,predict_val2))
predict_val1
```

```
       [1, 0],
       [1, 1],
       [0, 0],
       [1, 1],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [1, 0],
       [1, 1],
       [0, 0],
       [0, 0],
       [1, 1],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 1],
       [1, 1],
       [1, 0],
       [0, 0],
       [0, 0],
       [1, 0],
       [0, 1],
       [1, 0],
       [0, 0],
       [1, 1],
       [1, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [1, 1],
       [1, 1],
       [0, 0],
       [0, 0],
       [0, 0],
       [1, 1],
       [0, 0],
       [1, 1],
       [0, 1],
       [1, 0],
       [1, 1],
       [0, 1],
       [0, 0],
       [0, 1],
       [0, 0],
       [0, 0],
       [0, 0],
       [1, 1],
       [1, 1],
       [1, 1],
       [1, 0],
       [0, 0],
       [1, 0]])
```

```
predict_test1 = knn.predict(x_test)
predict_test2 = svm.predict(x_test)
```

```
predict_test1=np.column_stack((predict_test1,predict_test2))
predict_test1
```

```
        [1, 0],
        [0, 0],
        [1, 1],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [1, 1],
        [1, 0],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 1],
        [1, 1],
        [1, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [1, 1],
        [1, 1],
        [0, 0],
        [1, 1],
        [1, 0],
        [1, 0],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 0],
        [0, 0],
        [1, 1],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 0],
        [1, 0],
        [1, 0],
        [1, 1],
        [0, 0],
        [0, 0],
        [1, 0],
        [0, 0],
        [0, 0],
        [1, 1],
        [0, 0],
        [1, 1]])
```

```
rand_clf = RandomForestClassifier()
rand_clf.fit(predict_val1,val_test)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
rand_clf.score(predict_test1,y_test)
```

```
0.7402597402597403
```

```
grid_param = {
    "n_estimators" : [90,100,115],
    'criterion': ['gini', 'entropy'],
    'min_samples_leaf' :[1,2,3,4,5],
    'max_features' : ['auto','log2']
}
```

```
from sklearn.model_selection import GridSearchCV
grid_search = GridSearchCV(estimator=rand_clf, param_grid=grid_param, cv=3, n_jobs=-1,verbose=2)
```

```
grid_search.fit(predict_val1,val_test)
```

```
Fitting 3 folds for each of 60 candidates, totalling 180 fits
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
90 fits failed out of a total of 180.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
47 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in _validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validation.py", line 98, in validate_parameter_constrai
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' parameter of RandomForestClassifier must be an int

--------------------------------------------------------------------------------
43 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in _validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validation.py", line 98, in validate_parameter_constrai
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' parameter of RandomForestClassifier must be an int

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the test scores
       nan       nan       nan       nan       nan       nan       nan
       nan 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875
 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875
 0.7421875 0.7421875       nan       nan       nan       nan       nan
       nan       nan       nan       nan       nan       nan       nan
       nan       nan       nan 0.7421875 0.7421875 0.7421875 0.7421875
 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875 0.7421875
 0.7421875 0.7421875 0.7421875 0.7421875]
  warnings.warn(
```

▸ **GridSearchCV**

▸ **best_estimator_: RandomForestClassifier**

▾ RandomForestClassifier

```
RandomForestClassifier(max_features='log2', n_estimators=90)
```

```
grid_search.best_params_
```

```
{'criterion': 'gini',
 'max_features': 'log2',
 'min_samples_leaf': 1,
 'n_estimators': 90}
```

```
rand_clf = RandomForestClassifier(criterion= 'gini',
 max_features = 'sqrt',
 min_samples_leaf = 1,
 n_estimators = 90)
```

```
rand_clf.fit(predict_val1,val_test)
```

▾ RandomForestClassifier

```
RandomForestClassifier(n_estimators=90)
```

```
rand_clf.score(predict_test1, y_test)
```

```
0.7402597402597403
```